mediatools 1.1.0 – Handbook

Architecture, operation and safety design of shrink, progress, triage, flatten, sync and snapshot

Peter Wein

2026-11-03

Abstract

mediatools is a Python package (standard library only) with subcommands for maintaining large local media collections on Linux: `shrink` re-encodes videos to HEVC in place and records them in a per-directory SQLite database; `progress` reports the shrink status read-only; `triage` presents short clips in batches, one player window each, and removes the clips whose windows the user closed together with their hard links; `flatten` creates, per unit, a flat directory of hard links whose names encode the original location within the file-system name limit; `sync` copies a tree to a backup disk with rsync while preserving hard links, never deletes, and reports files that exist only on the target; `snapshot` keeps versioned rsnapshot backups on a removable disk, adopts an existing copy without copying data and refuses implausible runs after a dry-run probe. All machine-specific paths and names live in one local configuration file outside the repository. This handbook describes installation, configuration, the architecture (one class per module), the decision logic and safety rules of every subcommand, the backup strategy, testing and troubleshooting. All behaviour described is covered by the automated tests of the release; the snapshot workflow was additionally run end to end on an ext4 file system.

Contents

1 Overview **2**

2 Installation **3**
2.1 Prerequisites . . . 3
2.2 Clone and launcher . . . 3
2.3 Running the tests on the target machine . . . 3

3 Configuration **3**
3.1 Renaming existing databases . . . 4

4 Architecture **5**

5 shrink **7**
5.1 Invocation . . . 7
5.2 Decision per file . . . 8
5.3 Database . . . 9
5.4 Interruption and failures . . . 9

6 progress **9**

7 triage **9**
7.1 Workflow . . . 9
7.2 Why `_flat` links are removed too . . . 10
7.3 Recovering . . . 10

8 flatten **10**
8.1 Concept . . . 10
8.2 Naming . . . 11
8.3 Traceability . . . 11
8.4 Re-runs, replaced sources, pruning, undo . . . 11

9 sync **12**
9.1 Workflow . . . 12
9.2 rsync options . . . 12
9.3 Checks before the transfer . . . 13
9.4 Target-only report . . . 13
9.5 Both directions . . . 13
9.6 When to use sync and when snapshot . . . 13

10 snapshot **14**
10.1 Principle . . . 14
10.2 Commands . . . 16
10.3 Taking over an existing copy (adopt) . . . 16
10.4 Runs and levels . . . 17
10.5 Starting automatically when the disk is mounted . . . 17

11 Testing **17**

12 Troubleshooting **18**

13 Glossary **19**

14 Sources **19**

1 Overview

Subcommand	Purpose	Writes to
`shrink`	HEVC re-encode in place, name kept	original files (same inode), `shrink.db`
`progress`	which videos are recorded as done	data directory only
`triage`	review short clips, discard by closing the window	trash (or delete), kept list
`flatten`	flat hard-link view per unit	`<unit>/_flat/` only
`sync`	backup preserving hard links, no deletions	target directory, reports
`snapshot`	versioned backups with rsnapshot	snapshot root on the backup disk
`config`	show or create the configuration file	data directory only
`rename-db`	rename shrink databases to the configured name	database files only

Shared rules, implemented once in `mediatools.core`:

- `_flat`, `newShrinkedFiles` and hidden directories are never entered by a scan (`TreeWalker`, `MediaTypes.SKIP_DIRS`).
- Symbolic links are never followed or processed.
- Every run writes `logs/<command>_<timestamp>.log` into the data directory `$MEDIATOOLS_HOME` (default `~/scripts/mediatools.d`).

- External programs are called with argument lists, never through a shell, so file names with spaces, quotes or $ are safe.

2 Installation

2.1 Prerequisites

```
sudo apt install ffmpeg vlc libglib2.0-bin rsync rsnapshot
ffmpeg -hide_banner -encoders | grep libx265
```

`sudo` is needed because apt writes to `/usr` and the package database; from the user context it is sufficient, a root shell is not required. The second command runs as normal user and must print a line containing `libx265`.

2.2 Clone and launcher

The project lives in `~/Projects/mediatools`; the launcher is a small shell script in `~/scripts`, which is in `PATH`. Everything runs as normal user:

```
mkdir -p ~/Projects
git clone https://github.com/peddanet/mediatools.git ~/Projects/mediatools
cd ~/Projects/mediatools
./install.sh
mediatools --version
```

- `git clone URL DIR` – copy the repository into `DIR`.
- `install.sh` writes only `~/scripts/mediatools`; it refuses to overwrite a file it did not create.

2.3 Running the tests on the target machine

```
cd ~/Projects/mediatools
sudo apt install python3-pytest
python3 -m pytest -q
```

Expected: all tests pass. Tests that need ffmpeg, rsync or rsnapshot are skipped when the tool is missing.

3 Configuration

Everything that describes a concrete collection – paths, the database name, folder names worth abbreviating, the backup disk – is kept in one INI file outside the repository, so neither the code nor its history allows conclusions about the data:

```
$MEDIATOOLS_HOME/mediatools.ini      default: ~/scripts/mediatools.d/mediatools.ini
```

```
mediatools config --init        # write the commented template (never overwrites)
mediatools config --path        # print the file location
mediatools config               # print the effective values incl. defaults
```

Section / key	Default	Used by
`[collection] root`	–	every command called without a path

Section / key	Default	Used by
`[shrink] db_name`	`shrink.db`	shrink, progress, rename-db
`[flatten] abbreviations`	`Videos=V, Images=I, Audios=U`	flatten, naming level 1
`[snapshot] source, disk`	–	snapshot (required)
`[snapshot] root`	`rsnapshot`	directory on the disk
`[snapshot] retain_*`	alpha 7, beta 4, gamma 12, delta 2	snapshots per level; 0 disables a level
`[snapshot] exclude`	`newShrinkedFiles/`	rsync exclude patterns
`[snapshot] max_deletions`	500	abort threshold of a run

A missing or invalid value stops the command with a message that names the key and the expected format.

3.1 Renaming existing databases

If the shrink databases of an existing collection carry another file name, set `[shrink] db_name` and rename them in one pass. The old name is passed on the command line only:

```
mediatools rename-db OLDNAME.db --dry-run     # list, change nothing
mediatools rename-db OLDNAME.db               # rename after confirmation
```

`OLDNAME.db` and its backups `OLDNAME.db.bak_*` become `shrink.db` and `shrink.db.bak_*`. Renaming within a directory keeps the inode; content and time stamps are unchanged. A directory that already contains the new name is reported as a conflict and left alone – nothing is overwritten.

4 Architecture

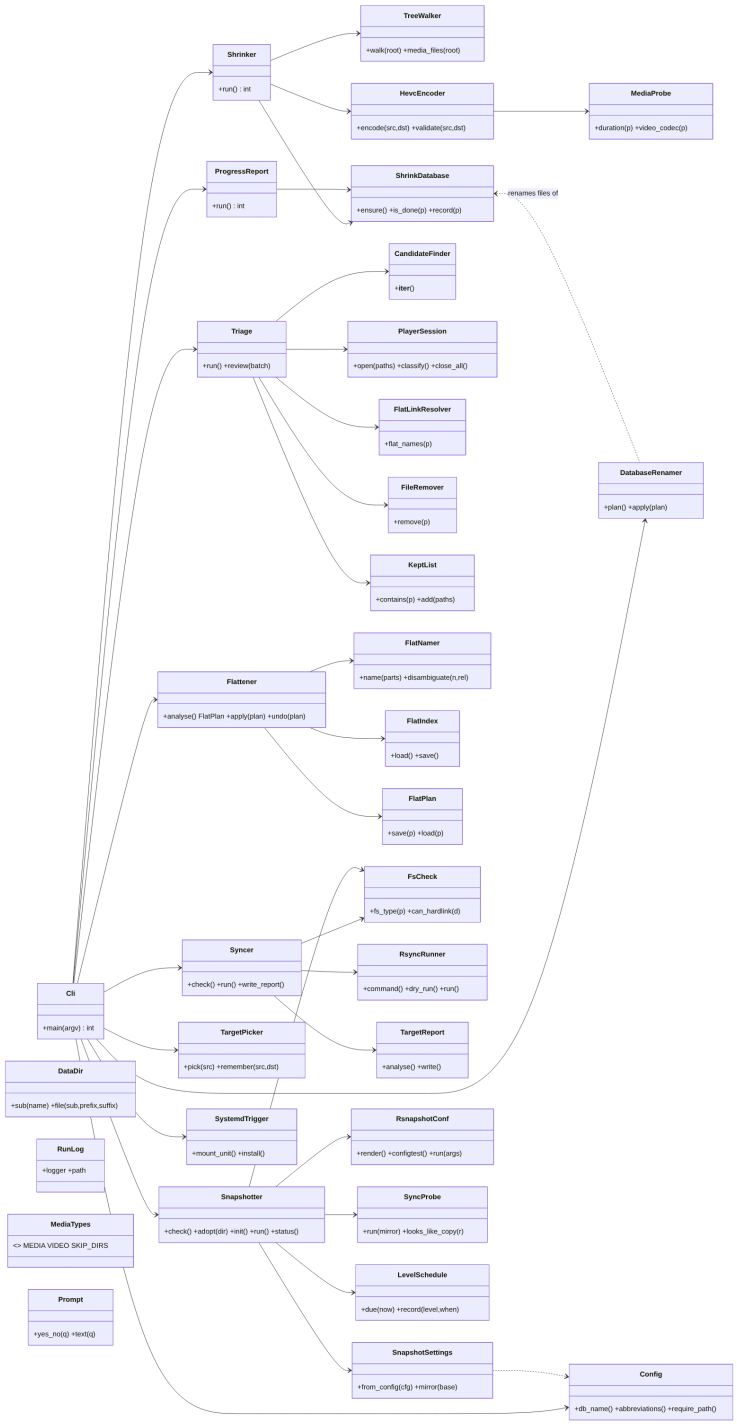


Figure 1: Classes and dependencies (one class per module)

```
src/mediatools/
  cli.py              Cli
  core/
    config.py         Config           local INI file (paths, names)
    datadir.py        DataDir          location of logs, state, plans, reports
    runlog.py         RunLog           logger to terminal and time-stamped file
    mediatypes.py     MediaTypes       extensions, reserved directory names
    probe.py          MediaProbe       ffprobe wrapper (duration, codec)
    walker.py         TreeWalker       sorted os.walk with skip rules
    prompt.py         Prompt           yes/no and text questions, --yes support
  shrink/
    database.py       ShrinkDatabase   per-directory database, schema migration
    dbrenamer.py      DatabaseRenamer  one-time renaming of databases
    encoder.py        HevcEncoder      ffmpeg call and result validation
    shrinker.py       Shrinker         per-file decision and copy-back
    progress.py       ProgressReport   read-only done/todo lists
  triage/
    candidates.py     CandidateFinder  lazy search by duration
    player.py         PlayerSession    one process per clip, classification
    linkresolver.py   FlatLinkResolver _flat names of a file (by inode)
    remover.py        FileRemover      gio trash or permanent delete
    keptlist.py       KeptList         decisions across sessions
    triage.py         Triage           batch workflow
  flatten/
    namer.py          FlatNamer        names within the byte limit
    flatindex.py      FlatIndex        _flat/_index.tsv
    plan.py           FlatPlan         actions, TSV persistence, undo input
    flattener.py      Flattener        analyse / apply / undo
  sync/
    fscheck.py        FsCheck          fs type, hard-link test, nesting
    target_picker.py  TargetPicker     interactive disk choice, remembered
    rsync_runner.py   RsyncRunner      rsync command, dry-run parsing
    target_report.py  TargetReport     target-only files, duplicate check
    syncer.py         Syncer           checks, dry run, transfer, report
  snapshot/
    settings.py       SnapshotSettings validated [snapshot] values
    rsnapconf.py      RsnapshotConf    generated rsnapshot.conf, configtest
    syncprobe.py      SyncProbe        rsync dry run: volume, removals
    schedule.py       LevelSchedule    which levels are due
    snapshotter.py    Snapshotter      check / adopt / init / run / status
    trigger.py        SystemdTrigger   user unit started by the disk mount
```

Design principles:

- **One responsibility per class**, every public class and method has a docstring (shown with `python3 -m pydoc mediatools.flatten.namer`).
- **Dependency injection** – logger, prompt, probe, player and remover are constructor arguments; the tests replace them with fakes.
- **Plan before action** where actions are irreversible or numerous (`flatten`, `sync`, `triage`): analysis, summary, confirmation, execution.

5 shrink

5.1 Invocation

```
mediatools shrink ~/Pictures/collection
mediatools shrink ~/Pictures/collection --keep-copies
mediatools shrink ~/Pictures/collection --crf 26 --threads 8 -y
```

Option	Default	Meaning
`--keep-copies`	off	do not touch originals; keep copies in `newShrinkedFiles/`; record nothing
`--crf N`	28	libx265 constant rate factor (lower = better quality, larger)
`--threads N`	6	ffmpeg threads
`--ext LIST`	`mp4`	extensions to process
`-y`	off	no confirmation question
`-v`	off	debug logging

5.2 Decision per file

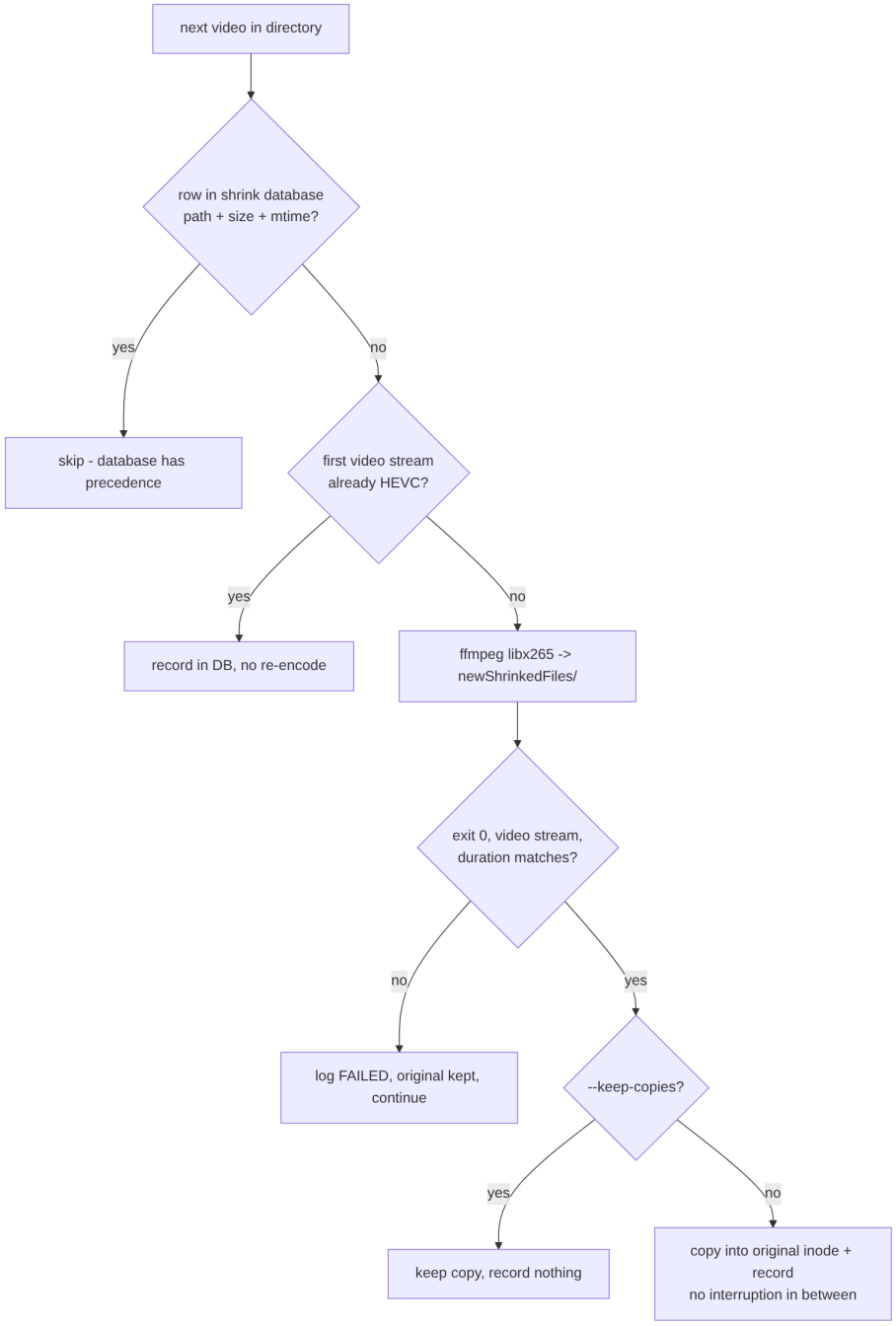


Figure 2: Decision logic of Shrinker

1. **Database first.** A row with identical path, size and mtime means “already processed”; the file is skipped without probing.
2. **Codec.** If the first video stream is already HEVC, the file is recorded but not re-encoded (re-encoding HEVC costs quality for little gain).
3. **Encode and validate.** `HevcEncoder` runs

```
ffmpeg -nostdin -y -threads 6 -i SRC -vcodec libx265 -crf 28 TMP
```

and accepts the result only if the exit code is 0, the output is not empty, has a video stream, and its duration equals the input within `max(1 s, 2 %)`.

4. **Copy back in place.** `shutil.copyfile` writes into the existing inode. All hard links of the file – `_flat` entries, deduplication links – therefore show the shrunk content immediately, and no second encode happens via another name.

5.3 Database

```
CREATE TABLE files (path text NOT NULL PRIMARY KEY,
                    size integer, modified text);
```

- One database per directory with videos; file name `[shrink] db_name` (default `shrink.db`).
- `modified` is the mtime after shrinking, format `YYYY-MM-DD HH:MM:SS`.
- Databases of the first script generation (no `PRIMARY KEY`) are backed up next to the original (suffix `.bak_` plus timestamp) and migrated in one transaction.

Inspection (normal user, read-only):

```
sqlite3 -readonly -header -column ~/Pictures/collection/A/Videos/shrink.db \
  "SELECT size, modified, path FROM files ORDER BY modified;"
```

5.4 Interruption and failures

Ctrl+C, Ctrl+Z and SIGTERM set a flag; the running ffmpeg receives the terminal's SIGINT too and stops; the original stays untouched and the temporary output is removed. Copy-back and database entry are executed as one unit and are never separated by an interruption. A file that cannot be encoded is logged as `FAILED`; the run continues and ends with exit code 2.

6 progress

```
mediatools progress ~/Pictures/collection
```

Writes `reports/progress_done_<ts>.txt` and `reports/progress_todo_<ts>.txt` and prints the counts. It creates no database and deletes nothing.

7 triage

7.1 Workflow

```
mediatools triage ~/Pictures/collection --dry-run
mediatools triage ~/Pictures/collection/A --batch 3
mediatools triage ~/Pictures/collection --min-seconds 10 --max-seconds 30
```

1. `CandidateFinder` walks the tree lazily and yields clips within the duration range that are not on the kept list.
2. `PlayerSession` starts one player process per clip (VLC muted and looping by default), then waits 3 s and flags processes that already ended.
3. The user closes the windows of clips to discard and presses Enter (`s` skips the batch, `q` quits).
4. Classification and summary, then confirmation:

Process state	Meaning	Action after y
running	keep	close window, add to kept list
ended with 0 after the start window	discard	remove file and its `_flat` links
ended within 3 s or with code ≠ 0	player error	keep, not added to kept list

7.2 Why `_flat` links are removed too

A hard link keeps the data alive. Removing only the original name of a clip while its `_flat` link still exists frees no space. `FlatLinkResolver` finds, by inode, all names of the file in the `_flat` directories of its ancestors; they are removed together. Hard links outside `_flat` (e.g. from a deduplication tool in another unit) are not touched; the summary shows `[!N further hard link(s) elsewhere: space not freed]`.

7.3 Recovering

```
gio trash --list
gio trash --restore "trash:///x.mp4"
```

Trashed files occupy space until the trash is emptied (`gio trash --empty`).

Option	Default	Meaning
`--max-seconds` / `--min-seconds`	60 / 0	duration range
`--batch`	10	windows per batch
`--player`	`vlc`	e.g. `mpv`, `"flatpak run org.videolan.VLC"`
`--audio`	off	play with sound
`--delete`	off	permanent deletion instead of trash
`--include-kept`	off	show kept clips again
`--dry-run`	off	list candidates only

8 flatten

8.1 Concept

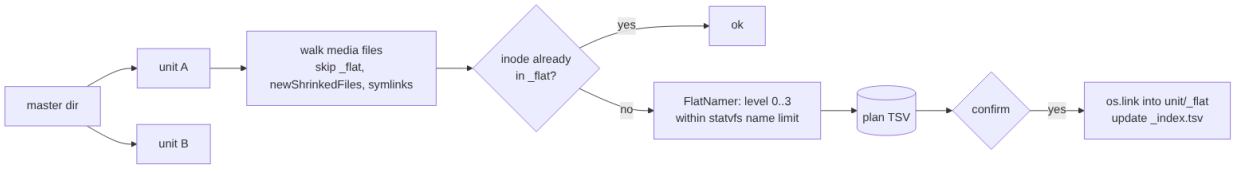


Figure 3: flatten workflow

The start directory is the **master level**; each direct sub-directory is a **unit**. Every media file below a unit (videos, images, audio; no databases) gets a hard link in `<unit>/_flat/`. Files lying directly in the unit are linked with their unchanged name, so `_flat` is complete.

```
mediatools flatten ~/Pictures/collection --dry-run
mediatools flatten ~/Pictures/collection
mediatools flatten ~/Pictures/collection --prune
mediatools flatten --undo \
  ~/scripts/mediatools.d/plans/flatten_2026-11-03_13-12-52.tsv
```

8.2 Naming

The limit is read at run time with `os.statvfs(unit).f_namemax` and counted in **bytes** (UTF-8). eCryptfs with encrypted file names reports 143 bytes on a lower file system with 255; ext4 and fscrypt report 255.

Level	Rule
0	path components joined with __, original file name unchanged
1	folder names from `[flatten] abbreviations` shortened (default `Videos` V, `Images` I, `Audios` U), `<id> <date> <time>` folders compacted to `<id>-<YYYYMMDD>T<hhmmss>`
2	level 1, then the front of the file stem is cut; the tail (usually an id) is kept; ~ marks the cut
3	directory part replaced by an 8-digit SHA-1 prefix, stem cut

Example of level 2 (limit 80 bytes, `abbreviations = posts=p, members=m, Videos=V`):

```
p__m__987654321-20240501T100000__V__~h_extended_commentary_and_extras_a1b2c3.mp4
```

In a simulation over a real collection of more than 100,000 media files with a limit of 143 bytes, more than 99.9 % of the names fitted at level 0; the rest needed level 1 or 2, none was impossible and no collision occurred. If two sources would get the same name, the second gets `~<6-digit hash>` before the extension.

The separator __ was chosen because it hardly ever occurs in real folder and file names; _ alone occurs almost everywhere.

8.3 Traceability

- `<unit>/_flat/_index.tsv` – one line `link name<TAB>source path relative to the unit`, written atomically.
- By inode, independent of any file:

```
find ~/Pictures/collection/A -samefile "$HOME/Pictures/collection/A/_flat/LINKNAME"
```

8.4 Re-runs, replaced sources, pruning, undo

Situation	Action
source inode already present in `_flat` (any name)	`ok`, nothing done
new source	`link`
same source path, new inode (file was replaced, e.g. re-downloaded)	`relink` (atomic: temporary link + rename)

Situation	Action
source gone, link still in `_flat`	`prune` – only with `--prune`
name impossible within the limit	`skip`, listed in the plan

Each run saves `plans/flatten_<ts>.tsv`. `--undo PLAN` removes exactly the links of that plan that are still identical to their source.

9 sync

9.1 Workflow

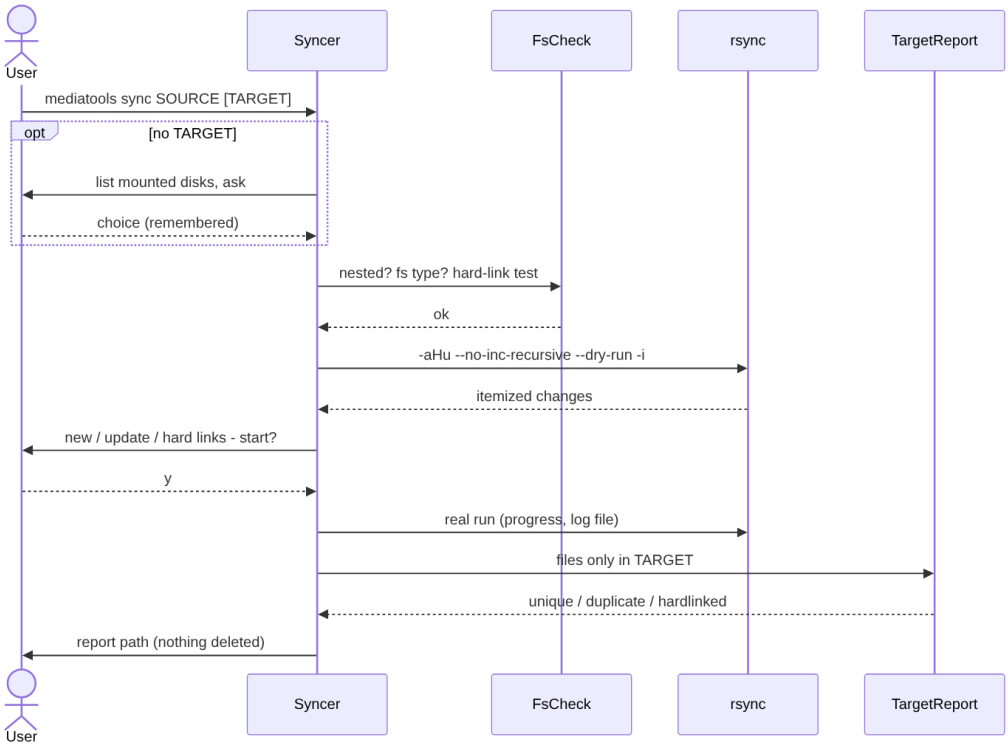


Figure 4: sync sequence

```
mediatools sync ~/Pictures/collection
mediatools sync ~/Pictures/collection /media/USER/backup/collection
mediatools sync ~/Pictures/collection --reverse
mediatools sync ~/Pictures/collection --report-only --find-duplicates
```

Without a target, mounted file systems below `/media`, `/run/media` and `/mnt` are listed with type and free space; the choice is stored in `state/sync_targets.json` and offered next time.

9.2 rsync options

```
rsync -aHu --no-inc-recursive --exclude=newShrinkedFiles/ --exclude=*.part \
      --exclude=.Trash-*/ --exclude=.mediatools-hl-* SOURCE/ TARGET/
```

Option	Reason
`-a`	archive mode: recursion, times, permissions, symlinks
`-H`	hard links stay hard links – `_flat` costs no space on the target
`-u`	a newer file on the receiver is never overwritten (protects shrunk files when copying back)
`--no-inc-recursive`	complete file list first; recommended by the rsync manual with `-H`
no `--delete`	files that exist only in the target are kept and reported

rsync detects hard links only inside the transfer set; therefore the unit directories and their `_flat` must be in the same run, which is the case when the master directory is synchronised as a whole.

9.3 Checks before the transfer

1. rsync installed, source is a directory.
2. Source and target do not contain each other.
3. File-system type of the target; exFAT, FAT, NTFS are refused.
4. Practical test: a temporary file is created and hard-linked in the target.
5. Dry run with `--itemize-changes`; summary of new files, updates, hard links, new directories; confirmation.

9.4 Target-only report

`reports/sync_target_only_<ts>.tsv`:

Status	Meaning
`unique`	only copy anywhere – keep unless known obsolete
`duplicate`	identical content elsewhere on the target (different inode)
`hardlinked`	another name of the same inode exists on the target

Content identity: equal size, then SHA-256 of first and last MiB, then full SHA-256. With `--find-duplicates` all groups of identical files in different inodes on the target are listed – space that hard-link deduplication could reclaim.

9.5 Both directions

`sync A B` copies A to B; `sync A --reverse` (or `sync B A`) copies back. Since nothing is deleted, copying back restores files that were removed on purpose (e.g. by `triage`); the dry-run summary shows how many files would be created – decline if that is not intended.

9.6 When to use sync and when snapshot

`sync` keeps one mirror and never deletes – safe, but deletions in the source accumulate on the disk. `snapshot` (next chapter) propagates deletions to the newest snapshot while older snapshots keep the files, so a mistaken deletion stays recoverable. On a disk that is used for snapshots, `sync` should no longer write into the snapshot root.

10 snapshot

10.1 Principle

rsnapshot stores every snapshot as a complete directory tree. Unchanged files are hard links to the previous snapshot, so a snapshot costs only the changed files plus directory metadata. mediatools generates the rsnapshot configuration from the INI file (`snapshot/rsnapshot.conf` in the data directory) and adds what rsnapshot does not do by itself: checks, a dry-run probe before every change, a deletion limit, adoption of an existing copy and a schedule for a disk that is not always attached.

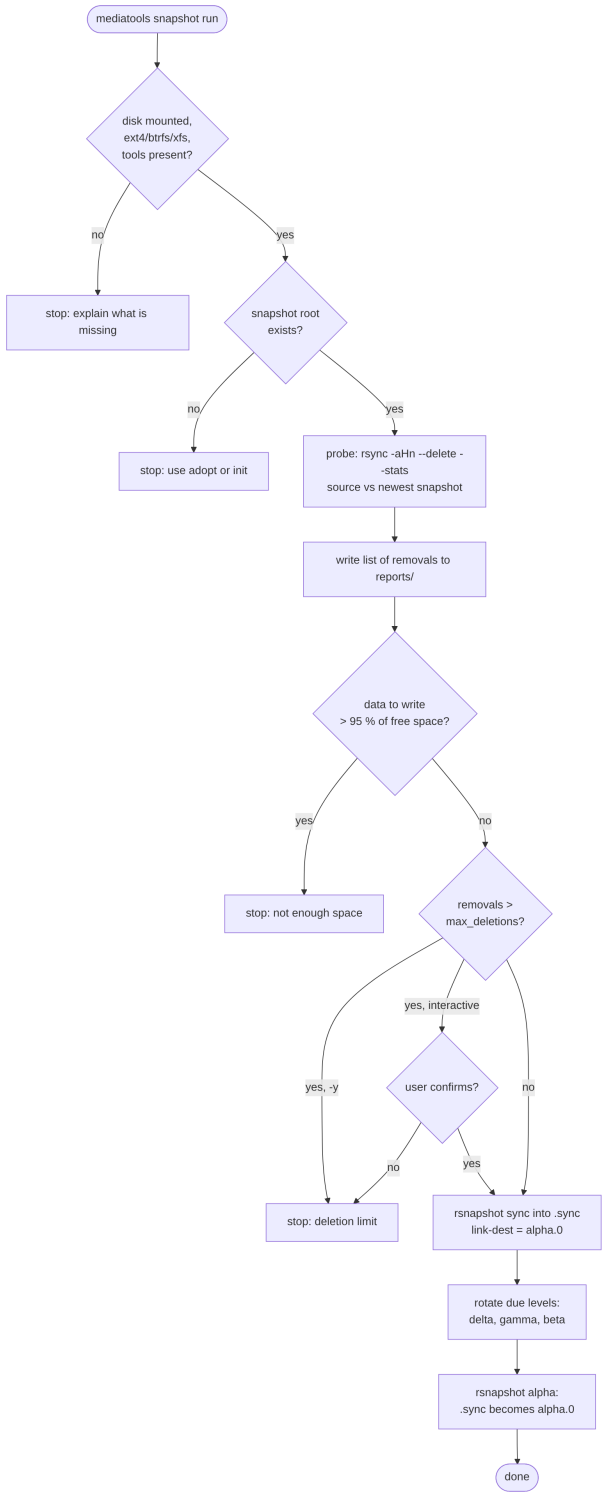


Figure 5: snapshot run

Fixed rsnapshot settings and why:

Setting	Reason
`no_create_root 1`	if the disk is not mounted, rsnapshot stops instead of filling the empty mount point on the system disk
`sync_first 1`	`rsnapshot sync` copies into `.sync`; the level commands only rotate
`link_dest 1`	unchanged files become hard links to the newest snapshot
`-aH --no-inc-recursive`	`_flat` hard links stay hard links within each snapshot
`--delete --delete-excluded`	only the newest snapshot follows deletions; older ones keep the files
`--relative`, destination `localhost/`	snapshot path is `<level>.<n>/localhost/<absolute source path>`

10.2 Commands

```
mediatools snapshot check              # read-only: config, disk, tools, probe
mediatools snapshot adopt DIR_ON_DISK  # once: existing copy = first snapshot
mediatools snapshot init               # or: empty start, first run copies all
mediatools snapshot run --dry-run      # probe only, removals listed in reports/
mediatools snapshot run                # probe, confirm, sync, rotate
mediatools snapshot run -y             # no questions; aborts instead of asking
mediatools snapshot status             # snapshots, free space, last rotations
mediatools snapshot install-trigger    # systemd user unit, started on mount
```

All commands run as normal user. rsnapshot is usually started as root by cron; with its own configuration file it can run as any user (rsnapshot manual). A normal user is sufficient as long as all files of the source belong to that user and the snapshot root on the disk is writable for him.

10.3 Taking over an existing copy (adopt)

If the disk already holds a copy of the source made with `sync`, that copy becomes the first snapshot without copying any data:

1. **Checks** – disk mounted, ext4/btrfs/XFS, directory on the same file system as the disk, snapshot root empty or missing.
2. **Probe** – `rsync -aHn --delete --stats` from the source to the given directory. If more than 25 % of the source volume would have to be transferred, the directory is not a copy of the source (wrong path or wrong level); the command stops and shows the top-level entries of both so the path can be corrected. The list of files that the first run would remove from the newest snapshot is written to `reports/`.
3. **Rename** – after confirmation the directory is renamed to `.sync/localhost/<source path>` (`rename(2)` within one file system: no data is moved, no space needed).
4. **Rotate** – `rsnapshot alpha` turns `.sync` into `alpha.0`.
5. **stand0** – `cp -al alpha.0 stand0` creates a hard-link copy that is never rotated away, so the files that the first runs remove stay available permanently. Omit with `--no-stand0`.

`stand0` must be a copy, not a renamed `alpha.0`: the next run links against `alpha.0`; without it, rsync would copy the whole source again.

10.4 Runs and levels

`snapshot run` does, in this order: checks; probe against the newest snapshot; refusal if the data to write exceed 95 % of the free space; warning and question (with `-y`: abort) if more than `max_deletions` files would leave the newest snapshot; `rsnapshot sync`; rotation of the due levels; `rsnapshot alpha`.

Level	Rotated	Kept (default)
`alpha`	every run	7
`beta`	7 days after its last rotation	4
`gamma`	30 days after its last rotation	12
`delta`	365 days after its last rotation	2

Because the disk is attached irregularly, the periods are measured from the last successful rotation (`state/snapshot_levels.json`), not from calendar dates. A higher level takes the oldest snapshot of the level below, so the first `beta` appears only once `alpha` has reached its retain count.

10.5 Starting automatically when the disk is mounted

```
mediatools snapshot install-trigger
systemctl --user daemon-reload
systemctl --user enable mediatools-snapshot.service
```

The command first asks the user's systemd instance whether the mount unit of `[snapshot] disk` is active (the disk must be mounted) and then writes `~/.config/systemd/user/mediatools-snapshot.service` with `WantedBy=<mount unit>`. The service runs `mediatools snapshot run -y` with idle I/O priority. Check afterwards:

```
systemctl --user status mediatools-snapshot.service
journalctl --user -u mediatools-snapshot.service -n 50
```

11 Testing

```
cd ~/Projects/mediatools
python3 -m pytest -q
PYTHONPATH=src:tests pylint src tests tools
```

File	Covers
`test_core.py`	media types, data directory, walker skip rules, prompt
`test_shrink.py`	database lookup, schema migration, full shrink cycle (HEVC skip, broken file, in-place inode, `_flat` skipped, DB precedence), keep-copies, duration validation, read-only progress
`test_triage.py`	kept list, player options, closed/open/early-exit classification with a stand-in player, `_flat` link resolution, removal of original plus link, no removal without confirmation
`test_flatten.py`	naming levels and byte limit, idempotency, relink, prune, undo, originals untouched

File	Covers
`test_sync.py`	hard-link test, rsync command without delete, itemize parsing, hard links preserved on the target, nothing deleted, duplicate classification, `-u` protection
`test_cli.py`	end-to-end runs of flatten, progress, sync and reverse sync
`test_config.py`	defaults, values and error messages of the INI file, commands without path, database renaming with conflicts
`test_snapshot.py`	settings validation, probe parsing, level schedule, generated rsnapshot.conf, systemd unit, refusal of a wrong directory, adoption without copying, deletion limit, removed files kept in `stand0`, hard links shared between snapshots, rotation of `beta`

The CI workflow (`.github/workflows/tests.yml`) runs all quality gates (`tools/check.sh`: format, lint, types, security, tests) on Python 3.10, 3.12 and 3.13. Architecture, UML diagrams and the API reference are in `docs/developer.md`.

12 Troubleshooting

Symptom	Cause	Action
`required tool(s) not found`	ffmpeg, rsync, gio or player missing	install the package named in the message
shrink: `FAILED … ffmpeg exit code`	input damaged or unsupported	`ffprobe -v error FILE`
sync refuses target: `cannot store hard links`	target is exFAT/FAT/NTFS	use ext4, btrfs or XFS
sync: `not writable`	target directory owned by root	see the command below
flatten: `other file system, skipped`	file on a different mount	hard links cannot cross file systems
triage: every clip `player error`	player exits immediately	start the player manually with one file
`no path given and [collection] root is not set`	no configuration	`mediatools config --init`, set the value
snapshot: `not a mount point`	disk not attached or wrong `disk` value	`lsblk -f` shows mount points
snapshot adopt: `does not look like a copy`	wrong directory or level	compare the two top-level listings in the message
snapshot run: `aborted because of the deletion limit`	many removals (or wrong `source`)	read the list in `reports/`, then run without `-y`

```
sudo install -d -o "$USER" -g "$USER" "/media/$USER/DISK/collection"
```

`install -d -o USER -g GROUP DIR` creates the directory with the given owner; it needs `sudo` because the mount point of a disk formatted as ext4 belongs to root. Afterwards everything runs as normal user.

13 Glossary

CRF – constant rate factor; quality-based rate control of x264/x265. FFmpeg H.265 guide

eCryptfs – stacked encrypting file system; with encrypted names the limit is 143 bytes. Kernel commit “eCryptfs: Improve statfs reporting”

ffprobe – FFmpeg tool that reports stream data. ffprobe documentation

Hard link – additional directory entry for the same inode; data exist once. link(1)

HEVC / H.265 – video coding standard, about half the bit rate of H.264 at comparable quality. ITU-T H.265

Inode – file-system object holding data and metadata; names point to it.

Itemize changes – rsync output describing each change with a code such as `>f+++++++++` or `hf`. rsync manual

libx265 – open-source HEVC encoder used by ffmpeg. x265 CLI documentation

mtime – last-modification time of a file.

rsync -H – preserve hard links within the transfer set. rsync manual: –hard-links

statvfs / f_namemax – system call reporting file-system limits, including the maximum name length.

Trash (FreeDesktop) – recoverable deletion area used by `gio trash`. Trash specification 1.0

UPSERT – `INSERT … ON CONFLICT … DO UPDATE` in SQLite; needs a unique key. SQLite UPSERT

rsnapshot – snapshot tool based on rsync and hard links. rsnapshot

Snapshot level – rsnapshot retention class (`alpha`, `beta`, …); each level keeps a fixed number of snapshots.

systemd user unit – service managed by the user’s own systemd instance, without root.

Unit – direct sub-directory of the master directory (e.g. one project).

14 Sources

Link check of 2026-11-03. “verified” = page content retrieved on that date.

Source	Status
rsync manual source, `--hard-links` and incremental recursion text	verified
rsync manual (samba.org)	official reference, not retrieved in this session
eCryptfs statfs patch, `ENC_NAME_MAX_BLOCKLEN_8_OR_16 143`	verified
SQLite UPSERT	verified
FFmpeg H.265 guide	verified via search index (site uses bot protection)
ffprobe documentation	verified
x265 CLI documentation	verified
ITU-T H.265 (01/26)	verified
FreeDesktop Trash specification 1.0	verified
VLC command-line help	verified
link(1), man7.org	verified
`gio help trash` (libglib2.0-bin 2.80, Ubuntu 24.04)	checked locally

Source	Status
rsync 3.2.7 behaviour (`-H`, `-u`, itemize codes)	checked by automated tests
rsnapshot manual page (mankier.com)	found via search; `sync_first`, `link_dest`, retain levels
rsnapshot	project home page, not retrieved in this session
rsnapshot(1), FreeBSD manual pages	found via search; running as a normal user with an own configuration file
rsnapshot 1.4.5 behaviour (adopt, rotation, link sharing)	checked end to end on ext4 and by automated tests